Universidad San Carlos de Guatemala
Facultad de ingeniería.
Ingeniería en ciencias y sistemas

ResiPark: Control de Accesos y Estacionamiento

PONDERACIÓN: 30 pts

Tiempo estimado: 67 hrs

Índice

1. MARCO FORMATIVO **3**
1.1. Valor 3
1.2. Competencia(s) 3
1.3. Habilidades blandas a formar 4
2. Resultado del Aprendizaje **4**
2.1. Objetivo SMART 4
3. Resumen Ejecutivo **5**
4. Enunciado del Proyecto **5**
4.1 Descripción del problema o necesidad a resolver 5
4.2 Alcance del proyecto 7
4.3 Recursos y herramientas a utilizar 11
4.4 Entregables 12
5. Material de apoyo 13
6. Metodología **14**
7. Cronograma **15**
8. Rúbrica de Calificación **16**
8.1 Requisitos para optar a la calificación 16
8.2 Detalle de la Calificación 17
8.3 Comentarios Generales 20

1. MARCO FORMATIVO

1.1. Valor

Nombre del valor	**¿Cómo se aplica en el proyecto?**
Responsabilidad	Planificar el desarrollo por etapas, avanzar de forma constante con commits incrementales y cumplir con todos los entregables en la fecha establecida.
Honestidad académica	Implementar personalmente cada estructura de datos, hilo y componente de la interfaz, sin copiar código de compañeros ni de terceros, y reconocer las fuentes consultadas.

1.2. Competencia(s)

Con la elaboración de este proyecto usted adquirirá la siguiente competencia:

- Diseña e implementa estructuras de datos dinámicas (lista simplemente enlazada, lista doblemente enlazada, lista circular, cola y pila) programadas desde cero en Java, sin recurrir al Java Collections Framework.
- Representa la distribución física de un parqueo mediante listas circulares de nodos y la visualiza gráficamente en Swing.
- Aplica concurrencia con Java Threads para simular dos garitas de ingreso y una de salida operando en paralelo, sincronizando el acceso a recursos compartidos mediante `synchronized`, `wait()` y `notify()`/`notifyAll()`.
- Construye interfaces gráficas de usuario utilizando exclusivamente Java Swing, codificadas manualmente, sin herramientas de diseño visual (drag and drop).
- Implementa persistencia de datos en archivos de texto plano y reconstruye las estructuras enlazadas a partir de ellos al iniciar la aplicación.
- Utiliza Git y GitHub aplicando un flujo de trabajo con commits incrementales y semánticos.
- Documenta técnica y funcionalmente el sistema desarrollado mediante manual técnico, manual de usuario e informe de desarrollo.

1.3. Habilidades blandas a formar

El proyecto le permitirá desarrollar las siguientes habilidades:

- **Pensamiento analítico y resolución de problemas:** descomponer la simulación en componentes (estructuras, reglas de negocio, hilos e interfaz) y resolver cada uno de forma incremental.
- **Gestión del tiempo y autoorganización:** planificar el trabajo por etapas para completar el proyecto dentro del tiempo estimado.
- **Pensamiento crítico y atención al detalle:** identificar casos límite, validaciones y condiciones de carrera, y corregirlos antes de la entrega.
- **Comunicación técnica:** explicar de forma oral y escrita el diseño y funcionamiento del sistema en la presentación en clase y en los manuales.

2. Resultado del Aprendizaje

2.1. Objetivo SMART

Al finalizar el proyecto, el estudiante habrá alcanzado el siguiente objetivo:

Específico (¿Qué?)	**Medible (¿Cuánto?)**	**Alcanzable (¿Cómo?)**	**Realista (¿Para qué?)**	**A Tiempo (¿Cuándo?)**
Desarrollar en Java un sistema de gestión de parqueo para un residencial que simule el ingreso, la permanencia y la salida de vehículos, usando estructuras de datos propias, hilos y una interfaz en Swing.	Seis estructuras implementadas manualmente (lista doble, lista simple, matriz ortogonal, dos colas y una pila), cinco garitas operando como hilos concurrentes, 150 espacios de parqueo representados y seis paneles de interfaz funcionales.	Aplicando los contenidos del curso sobre nodos y referencias, programación orientada a objetos, concurrencia y Swing, y siguiendo la metodología por etapas propuesta.	Para consolidar el dominio de estructuras de datos dinámicas y concurrencia mediante la simulación de un problema real, demostrable en clase.	Dentro de las 67 horas estimadas y antes de la fecha de entrega establecida en el cronograma.

3. Resumen Ejecutivo

El presente proyecto consiste en el desarrollo de un Sistema de Gestión de Parqueo para un Residencial, construido en Java con una interfaz gráfica en Java Swing. La aplicación no se limita a un CRUD: simula el flujo de los vehículos de residentes socios, residentes no socios y visitantes, desde que llegan a la entrada y son atendidos por una de dos garitas, hasta que reciben un espacio, permanecen estacionados y se retiran por la garita de salida.

Para lograrlo, el estudiante implementará manualmente una lista doblemente enlazada (residentes), una lista simplemente enlazada (vehículos de cada residente), dos listas circulares (espacios de cada área del parqueo), dos colas FIFO (entrada y salida) y una pila (historial de eventos). Las garitas funcionarán como hilos concurrentes sincronizados y la información de residentes y vehículos persistirá en archivos de texto plano.

4. Enunciado del Proyecto

4.1 Descripción del problema o necesidad a resolver

Un residencial necesita controlar el acceso, la permanencia y la salida de los vehículos de sus residentes y de los visitantes que recibe. El residencial cuenta con un parqueo dividido en dos áreas, dos garitas de entrada y una garita de salida, y requiere un sistema que administre la información de los residentes y sus vehículos, asigne espacios según el tipo de usuario y muestre en todo momento el estado del parqueo y de las filas de acceso.

Distribución del parqueo

- **Área de socios (A–C):** filas A, B y C con 25 espacios cada una (A1–A25, B1–B25, C1–C25), para un total de 75 espacios de uso exclusivo para residentes socios.
- **Área general/visitantes (E–I):** filas E, F, G, H e I con 15 espacios cada una (E1–E15, F1–F15, G1–G15, H1–H15, I1–I15), para un total de 75 espacios destinados a residentes no socios y visitantes.

Área	Filas	Espacios por fila	Identificadores	Total
Socios	A, B, C	25	A1-A25, B1-B25, C1-C25	75
General/Visitantes	E, F, G, H, I	15	E1–E15, F1–F15, …, I1–I15	75

No existe fila D. Cada espacio admite un único vehículo, sin importar su tipo.

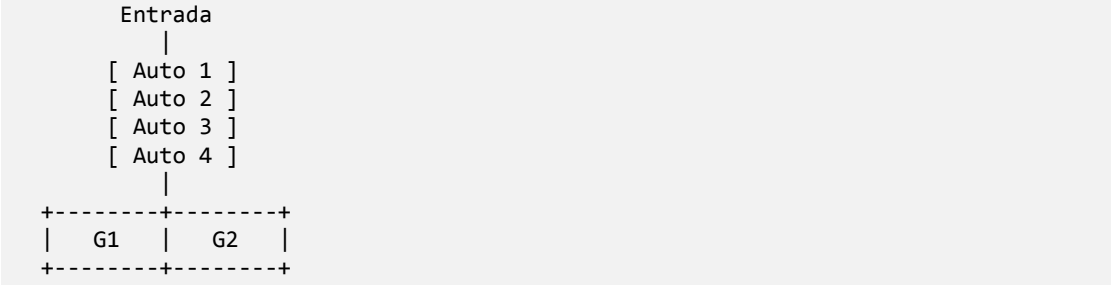
Reglas de asignación de espacios

Tipo de usuario	Área permitida
Residente socio	Primero se busca en el área de socios. Si está llena, puede ocupar un espacio del área general.
Residente no socio	Únicamente área general.
Visitante	Únicamente área general.

Si no hay un espacio disponible en las áreas permitidas para el vehículo, la garita lo **rechaza**: el vehículo no ingresa, vuelve al estado `FUERA` y se registra el evento correspondiente en la pila. Un vehículo rechazado no permanece esperando en la cola.

Garitas de entrada y salida

Existen **2 garitas de entrada** que atienden una única cola de entrada: los vehículos que llegan se forman en esa cola y cualquiera de las dos garitas toma al siguiente vehículo disponible. Existe **1 garita de salida** que atiende la cola de salida.



Estados de un vehiculo

Estado	Significado
`FUERA`	El vehículo no está en el residencial ni en ninguna cola.
`EN_COLA_ENTRADA`	Está esperando en la cola de entrada.
`ESTACIONADO`	Tiene un espacio asignado.
`EN_COLA_SALIDA`	Solicitó salir y espera en la cola de salida. Sigue ocupando su espacio.

Solo un vehículo en estado `FUERA` puede enviarse a la cola de entrada, y solo un vehículo `ESTACIONADO` puede solicitar salida. Esto impide que un mismo vehículo esté dos veces en una cola o en dos espacios.

Flujo de la simulación

El sistema debe reproducir el siguiente flujo para cada vehículo:

```
Usuario envía un vehículo a la entrada (panel de entrada)
       -> Cola de entrada (estado: EN_COLA_ENTRADA)
       -> Una de las 2 garitas de entrada lo atiende
       -> Se valida si es socio, no socio o visitante
       -> Se busca un espacio disponible según las reglas de asignación
              -> Si no hay espacio: se rechaza (estado: FUERA)
       -> Se asigna el espacio (estado: ESTACIONADO)
       -> El vehículo permanece estacionado
       -> El usuario solicita la salida (panel de salida)
       -> Cola de salida (estado: EN_COLA_SALIDA)
       -> La garita de salida lo atiende
       -> Se libera el espacio (estado: FUERA)
```

Cada paso relevante (ingreso a cola, asignación, rechazo, salida, liberación) se registra como un evento en la pila.

4.2 Alcance del proyecto

Alcance obligatorio:

El sistema deberá cumplir, como mínimo, con los siguientes requerimientos:

1. Lista doblemente enlazada – Residentes

Los residentes se almacenarán en una lista doblemente enlazada. Cada residente contendrá: ID, nombre, número de casa/lote, indicador de si es socio y su lista de vehículos. La lista debe permitir insertar, buscar por ID, eliminar, recorrer en ambos sentidos y actualizar residentes.

- El ID lo ingresa el usuario, debe ser único y no puede modificarse después del registro.
- Al eliminar un residente se eliminan también sus vehículos. No se permite eliminarlo si alguno de sus vehículos no está en estado `FUERA`.
- No se permite cambiar la condición de socio de un residente si alguno de sus vehículos no está en estado `FUERA`.

2. Lista simplemente enlazada – Vehículos

Los vehículos de cada residente se almacenarán en una lista simplemente enlazada propia de ese residente. Cada vehículo contendrá: placa, marca, modelo, color, tipo de vehículo (automóvil, motocicleta o pickup) y estado actual.

- Un residente puede tener de 0 a 3 vehículos; el sistema debe impedir el registro de un cuarto vehículo.

- La placa es única en todo el sistema, incluyendo las placas de visitantes que estén dentro del residencial o en cola.
- La lista debe permitir insertar, buscar por placa, eliminar y recorrer.

3. Listas circulares – Parqueo

Cada área del parqueo se representará mediante una lista circular simplemente enlazada de espacios: una lista para el área de socios (75 nodos, en orden A1…A25, B1…B25, C1…C25) y otra para el área general (75 nodos, en orden E1…E15, …, I1…I15). Cada nodo es un espacio físico, no un vehículo, y almacena:

- Identificador del espacio (por ejemplo, A12 o F3).
- Estado: libre u ocupado.
- Referencia al vehículo estacionado (o `null` si está libre).
- Tipo de espacio (socio o general).

Cada lista circular mantiene una referencia al último espacio asignado. La búsqueda de un espacio libre inicia en el nodo siguiente a esa referencia y recorre la lista **una sola vuelta**; si regresa al punto de inicio sin encontrar espacio, el área está llena. Esto reparte los vehículos a lo largo del parqueo en lugar de llenar siempre los primeros espacios.

El parqueo debe mostrarse gráficamente en Swing como una cuadrícula que respete la distribución física de la sección 4.1.

4. Cola de entrada

Cola FIFO implementada manualmente con nodos, en la que se forman los vehículos que llegan. Las dos garitas de entrada consumen vehículos de esta única cola.

5. Cola de salida

Segunda cola FIFO implementada manualmente para los vehículos que solicitan salir. La garita de salida consume vehículos de esta cola. El espacio del vehículo se libera cuando la garita lo atiende, no cuando entra a la cola.

6. Pila – Historial de eventos

Pila implementada manualmente con nodos que almacena el historial del sistema. Cada evento contiene: fecha y hora, tipo de evento, descripción y la garita involucrada (si aplica). Tipos de evento mínimos:

- Vehículo en cola de entrada
- Vehículo asignado a espacio (indicando el espacio)
- Vehículo rechazado (indicando el motivo)
- Vehículo en cola de salida
- Vehículo salió / espacio liberado
- Residente registrado, modificado o eliminado
- Vehículo registrado o eliminado
- Visitante registrado

El panel de eventos muestra la pila recorriéndola desde el tope (el evento más reciente primero). La pila no se persiste.

7. Hilos y sincronización

Cada garita funciona en su propio hilo:

- Garitas de entrada 1 y 2: consumen vehículos de la cola de entrada.
- Garita de salida: consume vehículos de la cola de salida.

La implementación debe demostrar el uso de:

- `Thread` o `Runnable`.
- `synchronized` para proteger las colas, las listas del parqueo y la pila de eventos.
- `wait()` cuando una cola está vacía, dentro de un ciclo `while`, y `notify()`/`notifyAll()` cuando se agrega un vehículo a la cola.
- Prevención de condiciones de carrera: dos garitas no pueden asignar el mismo espacio a vehículos distintos ni atender al mismo vehículo.

Cada garita debe simular un tiempo de atención con `Thread.sleep` (entre 1 y 3 segundos) para que la simulación sea observable. No se deben crear hilos únicamente para cumplir el requisito: cada hilo debe tener una función real dentro de la simulación.

8. Gestión de residentes y socios

El sistema debe permitir registrar, modificar, eliminar y buscar residentes; registrar y eliminar vehículos de un residente; mostrar los vehículos de un residente e identificar si es socio. Debe existir una validación que impida que un vehículo no autorizado ocupe un espacio del área de socios.

9. Visitantes

El sistema debe permitir registrar visitantes con: nombre, placa y el ID del residente al que visitan (que debe existir). Al registrarlo, su vehículo se envía a la cola de entrada y sigue el mismo flujo que los demás. Los visitantes son información temporal:

- Solo existen en memoria mientras están en cola o dentro del residencial.
- Al salir o ser rechazados se descartan.
- No se guardan como residentes ni se persisten en archivos.

10. Persistencia en archivos de texto

Los residentes y sus vehículos deben persistirse en archivos de texto plano, sin bases de datos. La información se guarda automáticamente al cerrar la ventana principal y, al iniciar, la aplicación reconstruye las listas enlazadas a partir de los archivos. Formato obligatorio:

```
residentes.txt:  ID|NOMBRE|CASA|SOCIO
vehiculos.txt:   PLACA|MARCA|MODELO|COLOR|ID_RESIDENTE
```

- `SOCIO` se guarda como `S` o `N`.
- Ningún campo puede contener el carácter `|`; el sistema debe validarlo al registrar.
- El estado de los vehículos no se persiste: al iniciar, todos están en estado `FUERA` y el parqueo arranca vacío.
- Si un archivo no existe, la aplicación inicia sin datos y lo crea al cerrar.
- Si una línea de `vehiculos.txt` hace referencia a un residente inexistente o supera el máximo de 3 vehículos, se omite y se registra un evento indicando la línea omitida.

11. Interfaz gráfica

La interfaz se desarrollará exclusivamente con Java Swing e incluirá, como mínimo, los siguientes paneles:

Panel	**Contenido mínimo**
Parqueo	Cuadrícula de los 150 espacios con su identificador, diferenciando libres y ocupados por color y mostrando la placa del vehículo estacionado. Se actualiza durante la simulación.
Residentes y vehículos	Registrar, buscar, modificar y eliminar residentes; registrar y eliminar vehículos de un residente y mostrarlos.
Entrada	Enviar a la cola un vehículo registrado (por placa) o registrar un visitante; mostrar la cola de entrada y el vehículo que atiende cada garita.
Salida	Seleccionar un vehículo estacionado para solicitar su salida; mostrar la cola de salida y el vehículo que atiende la garita.
Eventos	Historial de eventos de la pila, del más reciente al más antiguo.

Las actualizaciones de la interfaz realizadas desde los hilos de las garitas deben hacerse con `SwingUtilities.invokeLater`.

12. Restricciones

- Las listas, listas circulares, colas y pila deben implementarse manualmente con nodos.
- No se permite utilizar `ArrayList`, `LinkedList`, `Stack`, `Queue`, `Deque`, `HashMap` ni otras colecciones de `java.util`, ni arreglos, para almacenar los datos que corresponden a las estructuras solicitadas.
- Se permiten clases auxiliares de Java cuando sean necesarias: lectura y escritura de archivos, `String.split`, componentes de Swing (por ejemplo `JTable` con su modelo), aun cuando internamente usen colecciones, siempre que los datos se tomen de las estructuras propias.
- No utilizar JavaFX, bases de datos ni frameworks o librerías externas.
- El código debe organizarse en clases y paquetes de forma razonable, ser comprensible y evitar la sobreingeniería.

Alcance opcional:

Estos puntos no suman nota; sirven para demostrar mayor dominio en la calificación.

- **Control de la simulación:** pausar, reanudar y ajustar la velocidad de atención de las garitas.
- **Llegadas automáticas:** un hilo adicional que envía periódicamente a la cola de entrada vehículos registrados en estado `FUERA` y visitantes generados aleatoriamente, y que solicita salidas de vehículos estacionados.
- **Deshacer:** revertir la última operación de registro o eliminación de residentes o vehículos utilizando la pila de eventos.

4.3 Recursos y herramientas a utilizar

Tipo (Obligatorio / opcional)	Categoría (Software / hardware / Plataforma / Etc)	Descripción
Obligatorio	Software / Lenguaje	Java (JDK 17 o superior).
Obligatorio	Software / Librería gráfica	Java Swing, codificado manualmente (sin diseñadores drag and drop).
Obligatorio	Plataforma / Control de versiones	Git y GitHub, con commits incrementales y semánticos.
Obligatorio	Almacenamiento	Archivos de texto plano (.txt) para la persistencia de residentes y vehículos.
Opcional	Software / IDE	NetBeans, IntelliJ IDEA, Eclipse o VS Code.
Opcional	Software / Diagramación	draw.io, PlantUML o similar para el diagrama de clases y la arquitectura.

4.4 Entregables

A continuación se detalla cada uno de los elementos que deberá presentar el estudiante

Tipo	Descripción
Código fuente	Proyecto organizado en paquetes (por ejemplo: estructuras, modelo, hilos, persistencia y vista), alojado en un repositorio de GitHub con historial de commits incrementales. El repositorio con nombre: **IPC1A_NombreApellido_Carnet_Proyecto2**
Ejecutable	Archivo .jar ejecutable del sistema, junto con archivos de texto de ejemplo (residentes y vehículos) para cargar datos al iniciar.
Manual técnico	Arquitectura propuesta, diagrama de clases, estructuras de datos y su relación, flujo de entrada y salida de vehículos, uso de hilos y sincronización, organización de paquetes y formato de los archivos de persistencia.
Manual de usuario	Guía con capturas de pantalla de cada panel que explique cómo registrar residentes, vehículos y visitantes, ejecutar la simulación y consultar el historial de eventos.
Informe de desarrollo	Plan de implementación por etapas, registro del avance, problemas encontrados y soluciones aplicadas.
Presentación	Demostración en clase del funcionamiento de las listas, la matriz ortogonal, las colas, la pila, los hilos, la interfaz y la persistencia.

5. Material de apoyo

Categoría (Manual / Documentación oficial/ Ejemplo / Etc)	**Link**	**Descripción**
Documentación oficial	https://docs.oracle.com/javase/tutorial/uiswing/	Tutorial oficial de Java Swing: componentes, layouts y manejo de eventos.
Documentación oficial	https://docs.oracle.com/javase/tutorial/essential/concurrency/	Tutorial oficial de concurrencia en Java: hilos, sincronización y bloques protegidos (wait/notify).
Documentación oficial	https://docs.oracle.com/javase/tutorial/uiswing/concurrency/	Concurrencia en Swing y uso del Event Dispatch Thread.
Documentación oficial	https://docs.oracle.com/javase/tutorial/essential/io/	Lectura y escritura de archivos de texto en Java.
Manual	https://git-scm.com/book/es/v2	Libro Pro Git en español: uso de Git y flujo de trabajo con repositorios remotos.

6. Metodología

A continuación se presenta una metodología de la lógica que puedes seguir para elaborar el presente proyecto:

Pasos a seguir:

1. **Paso 1: Análisis y diseño**
   Leer el enunciado completo, identificar las entidades (residente, vehículo, visitante, espacio, evento) y elaborar la arquitectura, el diagrama de clases y la organización de paquetes antes de programar.
2. **Paso 2: Estructuras de datos base**
   Implementar los nodos, la lista simplemente enlazada, la lista doblemente enlazada, la cola y la pila. Probar cada estructura por consola de forma aislada.
3. **Paso 3: Matriz ortogonal**
   Construir la matriz enlazando los nodos en las cuatro direcciones según la distribución A–C (25 espacios) y E–I (15 espacios). Implementar la búsqueda de espacios libres por tipo y la asignación/liberación.
4. **Paso 4: Modelo y reglas de negocio**
   Implementar residentes, vehículos y visitantes, la validación del máximo de 3 vehículos y la restricción del área de socios.
5. **Paso 5: Persistencia**
   Implementar el guardado y la carga de residentes y vehículos en archivos de texto, reconstruyendo las listas enlazadas al iniciar.
6. **Paso 6: Hilos y sincronización**
   Crear los hilos de las garitas, sincronizar el acceso a las colas y a la matriz, e implementar la espera y notificación. Verificar que no se asignen espacios duplicados.
7. **Paso 7: Interfaz gráfica**
   Construir los seis paneles en Swing y conectarlos con la lógica, actualizando la vista a medida que avanza la simulación.
8. **Paso 8: Integración y pruebas**
   Probar el flujo completo con residentes, socios y visitantes, incluyendo casos límite: colas vacías, parqueo lleno y vehículos no autorizados.
9. **Paso 9: Documentación y entrega**
   Elaborar los manuales técnico y de usuario, el informe de desarrollo, generar el .jar y preparar la demostración.

Recomendaciones

- Implementar y probar cada estructura por separado antes de conectarla con la interfaz.
- Colocar `wait()` dentro de un ciclo `while` (no en un `if`) para volver a verificar la condición al despertar.
- Cuando un hilo de garita necesite actualizar la interfaz, hacerlo por medio de `SwingUtilities.invokeLater`, ya que Swing no es seguro para hilos.
- Agregar pausas (`Thread.sleep`) en la atención de las garitas para que la simulación sea observable durante la presentación.
- Realizar commits pequeños y frecuentes con mensajes descriptivos (por ejemplo, feat:, fix:, docs:).
- Priorizar un diseño claro y comprensible sobre soluciones innecesariamente complejas.

7. Cronograma

Tipo	Fecha Inicio	Fecha Fin
Publicación del enunciado	26/09/2026	26/09/2026
Desarrollo del proyecto	26/09/2026	23/10/2026
Entrega y calificación	24/10/2026	24/10/2026

8. Rúbrica de Calificación

En esta sección se aborda todo lo relacionado a la calificación, lo que permite claridad para el estudiante y el tutor académico de los aspectos a evaluar y los mismos que conocen desde el inicio por lo que todo proyecto debe incluirlos desde el inicio.

8.1 Requisitos para optar a la calificación

Antes de la evaluación del proyecto, debe cumplir con los requisitos que se indiquen en esta sección, de no cumplir usted tendrá una nota con valor de cero puntos.

Tema	**Descripción**	**Cumple (Sí/No)**
Estructuras propias	Listas, listas circulares, colas y pila implementadas manualmente con nodos; no se usan colecciones de `java.util` ni arreglos para sustituirlas.	
Tecnologías	Interfaz desarrollada exclusivamente con Java Swing codificado manualmente; sin JavaFX, bases de datos ni frameworks externos.	
Ejecución	El proyecto compila y el `.jar` se ejecuta correctamente a partir de los entregables.	
Repositorio	Repositorio de GitHub con historial de commits incrementales, entregado dentro de la fecha establecida.	
Originalidad	El código es de autoría propia. Las copias totales o parciales serán calificadas con cero puntos.	

8.2 Detalle de la Calificación

	Criterio/Nivel			**Nivel de evaluación**		
No.	**Criterio de evaluación**	**Punteo maximo**		**Satisfactorio 100% - 61%**	**Necesita mejorar 60% - 0%**	**Punteo Obtenido**
1	Habilidades	40				
1.1	Interfaz gráfica y visualización	15	Descripción	Los cinco paneles funcionan; el parqueo, las colas, las garitas y el historial se visualizan y actualizan durante la simulación, diferenciando claramente espacios libres y ocupados.	Faltan paneles, la interfaz no se actualiza durante la simulación o no permite distinguir el estado de los espacios.	
			Rango del punteo	15 – 10 pts	9 – 0 pts	
1.2	Simulación del flujo de vehículos	15	Descripción	El flujo completo (cola de entrada, garita, validación, asignación o rechazo, permanencia, cola de salida y liberación) se ejecuta sin errores para socios, no socios y visitantes.	El flujo se interrumpe, presenta errores o solo funciona para algunos tipos de usuario.	
			Rango del punteo	15 – 10 pts	9 – 0 pts	

1.3	Documentación y presentación	10	Descripción	Manuales completos y coherentes con el código; el estudiante explica con claridad su diseño y responde las preguntas del tutor.	Documentación incompleta o desactualizada; el estudiante no logra explicar el funcionamiento de su proyecto.	
			Rango del punteo	10 – 7 pts	6 – 0 pts	
	Sub-Total de Puntos	**40**				
2	Conocimientos	60				
2.1	Listas enlazadas (residentes y vehículos)	12	Descripción	La lista doble y la lista simple implementan todas las operaciones, respetan el máximo de 3 vehículos y la placa única, y mantienen correctamente sus referencias.	Faltan operaciones, se pierden referencias o se usan colecciones de Java en lugar de listas propias.	
			Rango del punteo	12 – 8 pts	7 – 0 pts	
2.2	Parqueo con listas circulares	15	Descripción	Los 150 nodos están enlazados en las cuatro direcciones, representan espacios físicos, se recorren para buscar espacios y se valida el área de socios.	La matriz se sustituye por arreglos, tiene enlaces incorrectos o permite asignaciones no autorizadas.	
			Rango del punteo	15 – 10 pts	9 – 0 pts	

2.3	Colas y pila	10	Descripción	Las colas FIFO de entrada y salida y la pila LIFO de eventos funcionan correctamente y se integran con la simulación.	Las estructuras no respetan el orden FIFO/LIFO o no se integran con la simulación.	
			Rango del punteo	10 – 7 pts	6 – 0 pts	
2.4	Hilos y sincronización	15	Descripción	Las tres garitas operan como hilos concurrentes con función real; se usan `synchronized`, `wait()` y `notify()`/`notifyAll()`, sin condiciones de carrera ni espacios asignados dos veces.	Los hilos no son concurrentes o no tienen función real, o se presentan bloqueos, pérdida de vehículos o asignaciones duplicadas.	
			Rango del punteo	15 – 10 pts	9 – 0 pts	
2.5	Persistencia en archivos	8	Descripción	Residentes y vehículos se guardan al cerrar con el formato indicado y las listas se reconstruyen al iniciar; los visitantes no se persisten.	No se guarda o no se reconstruye la información, o se persisten datos temporales de visitantes.	
			Rango del punteo	8 – 5 pts	4 – 0 pts	
	Sub-Total de Puntos	**60**				
	Total	**100**				

8.3 Comentarios Generales